



مستند جامع مهندسی نرم افزار، متدولوژی و معماری سیستم پلتفرم وبلاگ نویسی (Stand Blog)

فهرست مطالب (Table of Contents)

0. گروه ما

۱. مهندسی نیازمندی ها (Requirements Engineering)

- منابع نیازمندی
- نیازمندی های عملکردی (Functional Requirements)
- نیازمندی های غیر عملکردی (Non-Functional Requirements)
- اولویت بندی نیازمندی ها (MoSCoW)

۲. سناریوهای موارد کاربرد (Use Case Scenarios)

- UC-01: ثبت نام و ورود کاربر
- UC-02: بازیابی رمز عبور از طریق ایمیل (توکن امنیتی)
- UC-03: مشاهده لیست و جزئیات مقالات
- UC-04: ارسال پیام از طریق فرم تماس با ما

۳. متدولوژی توسعه نرم افزار (Methodology)

- انتخاب متدولوژی: اسکرام (Scrum)
- ساختار تیم اسکرام (Scrum Team)
- ساختار اسپرینت ها (Sprint Structure)
- گزارش تفصیلی اسپرینت ها

- ابزارهای مدیریت پروژه

۴. معماری و طراحی سیستم (Design & Architecture)

- نمای کلی معماری (High-Level Architecture)
- لایه‌های معماری (Architecture Layers)
- الگوهای طراحی پیاده‌سازی شده (Design Patterns)
- معماری پایگاه داده (Database Architecture)
- نمودار توالی (Sequence Diagram)
- نمودار کلاس‌ها (Class Diagram)
- نمودار مؤلفه‌ها (Component Diagram)

۵. راهنمای نصب و اجرا (Developer Manual)

- پیش‌نیازها (Prerequisites)
- مراحل نصب (Installation Steps)

۶. ساختار برنچ‌های Git (Git Branching Strategy)

- دیاگرام استراتژی برنچ‌ها (Git Flow)
- شرح برنچ‌ها
- گردش کار (Workflow)

۷. مستندات API (API Documentation)

۸. دیاگرام استقرار (Deployment Diagram)

۹. خلاصه معماری (Architecture Summary)

اطلاعات اعضای گروه و پروژه

- **اعضای گروه:** متین قاسمی، یوسف نظری، امیررضا ثبوتی بخش، رامبد رضوانپناه، عرفان حسینپور
- **محتویات فایل:** مستندات جامع مهندسی نرم افزار، متدولوژی توسعه نرم افزار، معماری و طراحی سیستم
- **تکنولوژی های به کار رفته:**

○ Python / Django : Backend

○ SQLite : Database

○ HTML5, CSS3, JavaScript, jQuery, Bootstrap : Frontend

○ **امکانات امنیتی:** CSRF Token ، هش کردن پسوردها ، توکن سازی امن جهت بازیابی رمز

هدف اصلی پروژه

ایجاد یک پلتفرم تعاملی مبتنی بر وب جهت انتشار مقالات و مطالب آموزشی-خبری، همراه با مدیریت کامل حساب های کاربری و امکان ارتباط مستقیم مخاطبان با مدیریت سایت .
پروژه دقیقاً چه کارهایی انجام می دهد؟

1. مدیریت و انتشار مقالات: (Article Management)

- نمایش جدیدترین مقالات در صفحه اصلی به همراه بنرها و اسلایدرهای متحرک .
- امکان مشاهده متن کامل هر مقاله همراه با تصویر شاخص، نام نویسنده و تاریخ انتشار .
- دسته بندی مطالب و نمایش آخرین مقالات در نوار کناری (Sidebar) تمامی صفحات .

2. مدیریت کاربران و امنیت: (Accounts & User Authentication)

- **ثبت نام و ورود:** ساخت حساب کاربری جدید و ورود به سیستم .
- **بازیابی رمز عبور پیشرفته:** ارسال لینک اختصاصی به همراه توکن امنیتی یکبار مصرف به ایمیل کاربر جهت بازنشانی رمز عبور .

3. ارتباط با کاربران: (Contact System)

- فرم «تماس با ما» برای دریافت پیام ها، پیشنهادات و نظرات کاربران (نام، ایمیل، موضوع، متن پیام) و ذخیره آن در دیتابیس .

4. صفحات معرفی: (Home & About Us)

- صفحه «درباره ما» جهت معرفی تیم یا اهداف وبلاگ .
- صفحه اصلی (Home) یکپارچه با اسلایدرها و آخرین مطالب سایت .

مهندسی نیازمندی‌ها (Requirements Engineering)

منابع نیازمندی :

مصاحبه و تعامل با کاربران بالقوه مثل دانشجویها و استادان و تمام کسانی که به نحوی میتوانند با این وبلاگ در تعامل باشند . الهام از نمونه های دیگر و تلاش بر بهبود عملکرد و کاستی های مدل های موجود .

نیازمندی های عملکردی (Functional Requirements) :

مدیریت کاربران :

- ثبت نام (register.html)
- ورود (Login.html)
- بازیابی رمز با توکن امنیتی

مدیریت مقالات :

- لیست مقالات
- جزئیات مقاله و نویسنده
- بارگزاری و بارگیری مقالات

: Contact App

- دریافت و ذخیره فرم تماس

About Us

توضیحات بیشتر :::

مدیریت کاربران: (account App)

- ثبت نام کاربران جدید: (register.html) ساخت حساب کاربر با اعتبارسنجی یکتایی ایمیل و نام کاربری .
- احراز هویت و ورود کاربران: (login.html) بررسی اعتبارنامه ها و ایجاد Session کاربر .
- بازیابی و بازنشانی رمز عبور: از طریق ارسال ایمیل حاوی توکن زمان دار و امن (tokens.py, password_reset_email.html).

مدیریت مقالات: (article App)

- نمایش لیست مقالات: (all_articles.html) نمایش لیست همراه با صفحه بندی. (Pagination)
- نمایش جزئیات مقاله: (article_detail.html) نمایش تصویر اصلی، نویسنده، تاریخ و متن کامل مقاله .
- نوار کناری: (sidebar.html, context_processor.py) نمایش دسته بندی ها و آخرین مقالات به صورت پویا در تمام صفحات .

ارتباط با ما: (contact App)

- ارسال فرم تماس: (contact.html, forms.py) دریافت نام، ایمیل، موضوع و متن پیام و ذخیره در جدول تماس دیتابیس .

درباره ما و صفحه اصلی: (home, about_us Apps)

- معرفی تیم و اهداف: (about_us.html) ارائه اطلاعات تیم توسعه و اهداف وبلاگ .
- صفحه اصلی: (index.html) ارائه بنرهای اسلایدر و نمایش آخرین مقالات .

نیازمندی‌های غیر عملکردی (Non-Functional Requirements) :

کارایی (Performance) : زمان بارگذاری صفحات کمتر از ۲ ثانیه با بهینه‌سازی فایل‌های ایستا (CSS/JS) و استفاده از Context Processors به جای کوئری‌های تکراری .

امنیت (Security) :

○ حفاظت در برابر حملات XSS و SQL Injection توسط Django ORM و Template Engine.

○ استفاده الزامی از CSRF Token در تمامی فرم‌ها .

○ هش کردن پسوردها با الگوریتم استاندارد . PBKDF2 (SHA256)

قابلیت نگهداری (Maintainability) : ساختار ماژولار (App-based) در جنگو جهت توسعه مستقل ماژول‌ها توسط اعضای تیم .

مقیاس‌پذیری (Scalability) : جداسازی لایه پایگاه داده و قابلیت سوئیچ از SQLite به PostgreSQL بدون تغییر در کدهای منطق برنامه.

اولویت بندی نیازمندی ها :

MUST HAVE

شناسه	نیازمندی	دلیل
FR-01	ثبت‌نام کاربر (Register)	هسته اصلی سیستم احراز هویت
FR-02	ورود کاربر (Login)	بدون آن، مدیریت کاربران ممکن نیست
FR-03	نمایش لیست مقالات در صفحه اصلی	مهمترین کاربرد سایت برای مخاطب
FR-04	نمایش جزئیات کامل مقاله	هدف اصلی وبلاگ (مطالعه محتوا)
FR-05	نوار کناری پویا (Sidebar) با دسته‌بندی و آخرین مقالات	ناوبری سریع و بهبود UX
FR-06	فرم تماس با ما (Contact)	ارتباط مستقیم با مدیریت
FR-07	امنیت پایه (CSRF, SQL Injection, XSS)	نیازمندی غیرعملکردی حیاتی
FR-08	هش کردن رمز عبور (PBKDF2)	امنیت حساب‌های کاربری

● SHOULD HAVE (بسیار مهم - اما قابل انتشار در نسخه بعدی)

شناسه	نیازمندی	دلیل
FR-09	بازیابی رمز عبور با توکن امنیتی ایمیل	بهبود تجربه کاربری، اما نسخه اولیه می‌تواند با سوال امنیتی جایگزین شود
FR-10	در لیست مقالات (Pagination) صفحه‌بندی	برای تعداد بالای مقالات ضروری است، اما در ابتدا با ۱۰ مقاله قابل چشم‌پوشی است
FR-11	نمایش اسلایدر در صفحه اصلی	زیبایی و جذابیت، اما محتوای اصلی همان لیست مقالات است
FR-12	ذخیره فرم تماس در دیتابیس	ضروری برای پیگیری، اما می‌توان در نسخه اولیه فقط ایمیل شود

● COULD HAVE (امتیاز مثبت - در صورت وقت و منابع)

شناسه	نیازمندی	دلیل
FR-13	صفحه «درباره ما» با معرفی تیم	اطلاعات تکمیلی، اما روی عملکرد اصلی تأثیری ندارد
FR-14	نمایش نام نویسنده و تاریخ در جزئیات مقاله	ارزش افزوده، اما محتوای مقاله اصل است
FR-15	Sidebar برای Context Processor استفاده از	بهینه‌سازی عملکرد، اما با کوئری مستقیم هم کار می‌کند
FR-16	PostgreSQL به SQLite قابلیت سوییچ از	مقیاس‌پذیری، اما برای نسخه اولیه نیازی نیست

⊖ WON'T HAVE (فعلاً انجام نمی‌شود)

شناسه	نیازمندی	دلیل
FR-17	ویرایش مقاله توسط کاربران عادی	فقط مدیر باید دسترسی داشته باشد (در Scope پروژه نیست)
FR-18	سیستم نظرخواهی (Comment)	در نیازمندی‌های اولیه نیامده است
FR-19	پنل مدیریت پیشرفته (Admin Panel سفارشی)	Django Admin پیش‌فرض کافی است
FR-20	API عمومی برای دریافت مقالات	خارج از هدف پروژه (صرفاً وب سایت است)
FR-21	ثبت‌نام با شماره موبایل	فقط ایمیل پشتیبانی می‌شود

Use Case Senarios

UC-01: ثبت نام و ورود کاربر

بازیگر اصلی: کاربر جدید / کاربر عضو

پیش شرط: کاربر به اینترنت و مرورگر دسترسی دارد

◆ سناریوی اصلی (مسیر خوش بخت)

مرحله	اقدام بازیگر	پاسخ سیستم
۱	کاربر روی لینک «ثبت نام» کلیک می کند	بارگذاری می شود register.html صفحه
۲	کاربر فرم را با نام، ایمیل، رمز عبور پر می کند	اعتبارسنجی سمت کلاینت انجام می شود
۳	کاربر دکمه «ثبت نام» را می زند	سیستم یکتایی ایمیل و نام کاربری را بررسی می کند
۴	(اطلاعات معتبر است)	سیستم حساب کاربری را ایجاد و رمز را هش می کند
۵	-	کاربر به صفحه ورود هدایت می شود با پیام موفقیت
۶	کاربر ایمیل و رمز خود را در فرم ورود وارد می کند	سیستم اعتبارنامه ها را بررسی می کند
۷	-	ایجاد می کند و کاربر را به صفحه Session سیستم اصلی می برد

◆ سناریوی جایگزین (خطاها و استثناها)

مرحله	شرط خطا	واکنش سیستم
الف-۳	ایمیل یا نام کاربری تکراری است	«پیام خطا: «این ایمیل قبلاً ثبت شده است
ب-۳	رمز عبور کمتر از ۸ کاراکتر است	«پیام خطا: «رمز عبور باید حداقل ۸ کاراکتر باشد
الف-۶	ایمیل یا رمز اشتباه است	«پیام خطا: «نام کاربری یا رمز عبور نامعتبر است
ب-۶	کاربر ۳ بار تلاش ناموفق داشته	سیستم قفل موقت ۵ دقیقه ای اعمال می کند
ج-۶	کاربر رمز را فراموش کرده	هدایت می شود (بازیابی رمز) UC-02 به

UC-02: بازیابی رمز عبور از طریق ایمیل (توکن امنیتی)

بازیگر اصلی: کاربر عضو (رمز فراموش شده)

پیش شرط: کاربر قبلاً ثبت نام کرده و ایمیل خود را دارد

◆ سناریوی اصلی

مرحله	اقدام بازیگر	پاسخ سیستم
۱	کاربر در صفحه ورود، روی «رمز عبور را فراموش کرده ام» کلیک می کند	صفحه درخواست بازیابی رمز باز می شود
۲	کاربر ایمیل خود را وارد می کند	سیستم بررسی می کند که ایمیل در دیتابیس وجود دارد
۳	-	سیستم یک توکن یکبار مصرف با زمان انقضا (مثلاً ۱ ساعت) تولید می کند
۴	-	سیستم یک ایمیل حاوی لینک بازیابی به کاربر ارسال می کند
۵	کاربر روی لینک کلیک می کند	سیستم توکن را اعتبارسنجی می کند (معتبر و منقضی نشده)
۶	-	صفحه تنظیم رمز جدید نمایش داده می شود
۷	کاربر رمز جدید را وارد و تأیید می کند	سیستم رمز جدید را هش و ذخیره می کند
۸	-	کاربر به صفحه ورود هدایت می شود با پیام موفقیت

◆ سناریوی جایگزین

مرحله	شرط خطا	واکنش سیستم
الف-۲	ایمیل در سیستم وجود ندارد	پیام: «این ایمیل در سیستم ثبت نشده است» (اما برای امنیت، پیام کلی نشان داده می شود)
الف-۴	ارسال ایمیل با شکست مواجه شود	«پیام: «خطا در ارسال ایمیل، لطفاً دوباره تلاش کنید»
الف-۵	توکن منقضی شده باشد (بیش از ۱ ساعت)	پیام: «لینک منقضی شده است، دوباره درخواست دهید» و بازگشت به مرحله ۱
ب-۵	توکن معتبر نباشد (دستکاری شده)	پیام: «لینک نامعتبر است» و کاربر به صفحه اصلی هدایت می شود
الف-۷	رمز جدید با تأیید مطابقت نداشته باشد	پیام خطا و امکان اصلاح

UC-03: مشاهده لیست و جزئیات مقالات

بازیگر اصلی: بازدیدکننده / کاربر عضو
پیش شرط: هیچ (دسترسی عمومی)

◆ سناریوی اصلی

مرحله	اقدام بازیگر	پاسخ سیستم
۱	(index.html) کاربر وارد صفحه اصلی می‌شود	سیستم جدیدترین مقالات را با اسلایدر و لیست نمایش می‌دهد
۲	کاربر روی عنوان یک مقاله کلیک می‌کند	هدایت article_detail.html سیستم به صفحه می‌کند
۳	-	سیستم متن کامل، تصویر شاخص، نام نویسنده و تاریخ را نمایش می‌دهد
۴	برای (Sidebar) کاربر از نوار کناری مشاهده دسته‌بندی استفاده می‌کند	سیستم مقالات آن دسته را فیلتر و نمایش می‌دهد
۵	کاربر از صفحه‌بندی برای دیدن مقالات قدیمی‌تر استفاده می‌کند	سیستم صفحه بعدی مقالات را بارگذاری می‌کند

◆ سناریوی جایگزین

مرحله	شرط خطا	واکنش سیستم
۲- الف	مقاله با آن آدرس وجود ندارد	«خطای ۴۰۴ با پیام «مقاله یافت نشد»
۲- ب	است و کاربر مدیر (Draft) مقاله در حالت پیش‌نویس نیست	«پیام «این مقاله در دست انتشار نیست»
۴- الف	دسته‌بندی انتخابی مقاله‌ای نداشته باشد	پیام «هیچ مقاله‌ای در این دسته یافت نشد»
۵- الف	صفحه درخواستی از تعداد کل صفحات بیشتر باشد	کاربر به صفحه آخر هدایت می‌شود

UC-04: ارسال پیام از طریق فرم تماس با ما

بازیگر اصلی: بازدیدکننده / کاربر عضو

پیش شرط: کاربر به اینترنت دسترسی دارد (نیاز به لاگین ندارد)

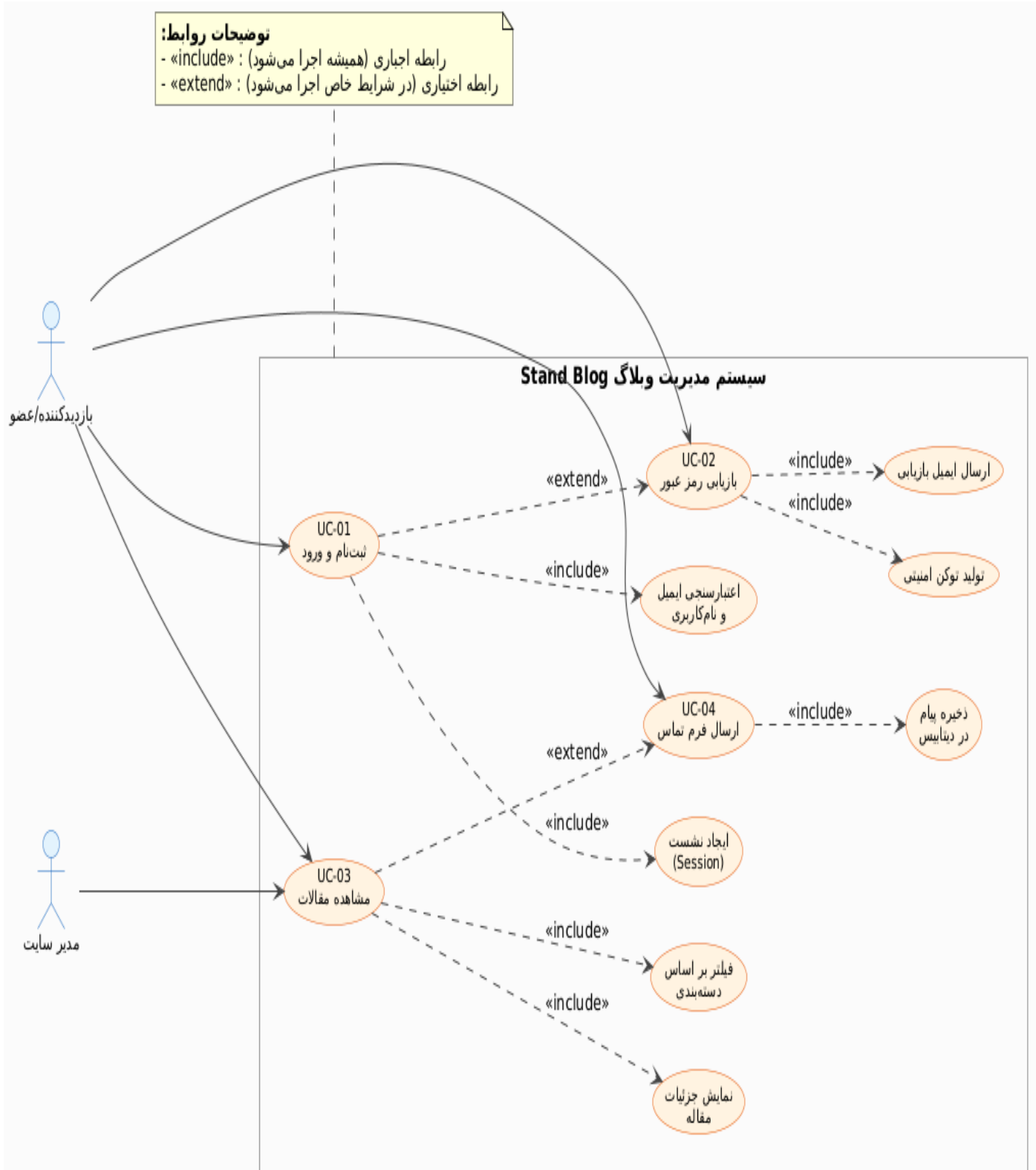
سناریوی اصلی

مرحله	اقدام بازیگر	پاسخ سیستم
۱	کاربر روی لینک «تماس با ما» کلیک می‌کند	بارگذاری می‌شود <code>contact.html</code> صفحه
۲	کاربر فرم را با نام، ایمیل، موضوع و متن پیام پر می‌کند	اعتبارسنجی اولیه (همه فیلدها پر شده باشند)
۳	کاربر دکمه «ارسال» را می‌زند	را بررسی می‌کند CSRF Token سیستم
۴	-	سیستم اطلاعات را در دیتابیس (جدول تماس) ذخیره می‌کند
۵	-	پیام موفقیت «پیام شما با موفقیت ارسال شد» به کاربر نشان داده می‌شود
۶	-	مدیر سایت در پنل مدیریت، پیام جدید را مشاهده می‌کند

سناریوی جایگزین

مرحله	شرط خطا	واکنش سیستم
۲-الف	ایمیل معتبر نیست (فرمت اشتباه)	«پیام خطا: لطفاً یک ایمیل معتبر وارد کنید»
۲-ب	یکی از فیلدها خالی است	«پیام خطا: همه فیلدها اجباری هستند»
۳-الف	نامعتبر است CSRF Token	خطای ۴۰۳ و درخواست مجدد
۴-الف	ذخیره در دیتابیس با خطا مواجه شود	«پیام: خطا در ارسال پیام، لطفاً دوباره تلاش کنید»
۴-ب	کاربر ظرف ۵ دقیقه بیش از ۳ پیام ارسال کرده باشد	و پیام: «شما بیش از حد (Rate Limiting) محدودیت نرخ مجاز پیام ارسال کرده‌اید، کمی صبر کنید»

: Usecase Diagram



متدولوژی توسعه نرم افزار (Methodology)

1. پروژه Stand Blog با استفاده از چارچوب چابک Scrum مدیریت شده است. دلایل انتخاب این متدولوژی عبارتند از:

- تغییرپذیری نیازمندی ها : امکان تغییر اولویت ها در هر اسپرینت بدون تأثیر منفی بر کل پروژه
- تحویل تدریجی : هر اسپرینت یک خروجی عملیاتی (Potentially Shippable) تولید می کند
- شفافیت : برگزاری جلسات هفتگی و بازبینی اسپرینت (Sprint Review)
- مدیریت ریسک : شناسایی زودهنگام مشکلات از طریق بازخورد مداوم
- تیم ۵ نفره : ساختار اسکرام برای تیم های کوچک تا متوسط (۳-۹ نفر) بهینه است

2 . ساختار تیم اسکرام (Scrum Team)

تیم پروژه شامل ۵ عضو با نقش های مشخص زیر است:

نقش	مسئولیت ها	نفر مسئول
مالک محصول (Product Owner)	تعریف نیازمندی ها، تأیید خروجی اسپرینت Product Backlog تعیین اولویت های	(Classified)
اسکرام مستر (Scrum Master)	تسهیل فرآیندهای اسکرام، رفع موانع، برگزاری جلسات	همه اعضا (چرخشی)
توسعه دهنده بک اند و فرانت اند	یکپارچه سازی UI/UX پیاده سازی منطق برنامه، طراحی	متین قاسمی
و گیت هاب DevOps مدیر	استقرار، کنترل نسخه CI/CD مدیریت مخزن،	یوسف نظری
مستندسازی و مهندسی نیازمندی ها	تهیه مستندات، تحلیل نیازمندی ها، انتخاب متدولوژی	امیررضا ثبوتی بخش
کیفیت کد و تست	نوشتن تست ها، تحلیل کیفیت کد، رعایت استانداردها	رامبد رضوان پناه
ارائه و یکپارچه سازی	ارائه پروژه، یکپارچه سازی ماژول ها	عرفان حسین پور

3. ساختار اسپرینت‌ها (Sprint Structure)

هر اسپرینت در این پروژه به مدت ۲ هفته برنامه ریزی شده است.

جلسه	زمان	توضیح
Sprint Planning	شروع هر اسپرینت (۲-۴ ساعت)	انتخاب آیتم‌های Product Backlog برای اسپرینت جاری و تبدیل به Sprint Backlog
Daily Stand-up	روزانه (۱۵ دقیقه)	پاسخ به ۳ سوال: هفته قبل چه کردم؟ چه خواهم کرد؟ چه مانعی دارم؟
Sprint Review	پایان اسپرینت (۱-۲ ساعت)	نمایش خروجی اسپرینت به ذی‌نفعان و دریافت بازخورد
Sprint Retrospective	پایان اسپرینت (۱ ساعت)	بررسی فرآیندها، شناسایی نقاط قوت و بهبود

4. گزارش تفصیلی اسپرینت‌ها

اسپرینت ۱: پایه‌سازی و معماری (هفته ۱ و ۲)

هدف: ایجاد زیرساخت‌های اولیه پروژه برای توسعه در اسپرینت‌های بعدی

فعالیت	شرح	خروجی
Sprint Planning	تعریف اهداف اسپرینت، انتخاب آیتم‌های اولیه از Product Backlog	Sprint Backlog
تعریف مدل داده‌ای	طراحی مدل‌های اولیه شامل User, Article, Contact, Category	فایل models.py
تنظیم پروژه جنگو	ایجاد پروژه Django، تنظیمات Settings.py، اتصال به دیتابیس	پروژه اولیه Django
مدیریت برنج‌ها	ایجاد برنج‌های اصلی (main, develop, feature/*)	ساختار گیت‌هاب
قالب بندی پایه	طراحی base.html با ساختار Header, Footer, Sidebar	فایل‌های تمپلیت

خروجی‌های اسپرینت ۱: (Deliverables)

- معماری پایه پروژه (MVC)
 - قالب‌بندی base.html با بخش‌های اصلی
 - ساختار اولیه دیتابیس (مدل‌های User, Article, Contact, Category)
 - مخزن گیت‌هاب با برنچ‌بندی استاندارد (GitFlow)
- اسپرینت ۲: پیاده‌سازی هسته سیستم (هفته ۳ و ۴)
- هدف: پیاده‌سازی قابلیت‌های اصلی و اساسی پروژه

خروجی	شرح	فعالیت
Sprint Backlog مصوب	انتخاب آیتم‌های اولویت‌دار	Sprint Planning
اپلیکیشن account	پیاده‌سازی ثبت‌نام، ورود، خروج، و بازیابی رمز با توکن	سیستم احراز هویت
اپلیکیشن article	ایجاد مدل مقاله، نمایش لیست و جزئیات، Sidebar پویا	مدیریت مقالات
اپلیکیشن contact	پیاده‌سازی فرم تماس با اعتبارسنجی و ذخیره در دیتابیس	فرم تماس
ارسال ایمیل‌های تراکنشی	پیگیری‌بندی SMTP و ارسال لینک بازیابی رمز عبور	ارسال ایمیل
فایل‌های tests.py	نوشتن تست‌های واحد برای مدل‌ها و ویوهای اصلی	تست‌های اولیه

خروجی‌های اسپرینت ۲: (Deliverables)

- سیستم کامل احراز هویت (ثبت‌نام، ورود، خروج، بازیابی رمز)
- ارسال ایمیل حاوی توکن امنیتی برای بازنشانی رمز عبور
- لیست مقالات با صفحه‌بندی و نمایش جزئیات کامل
- فرم تماس با کاربر با اعتبارسنجی و ذخیره در دیتابیس
- نوار کناری (Sidebar) پویا در تمام صفحات

5. ابزارهای مدیریت پروژه

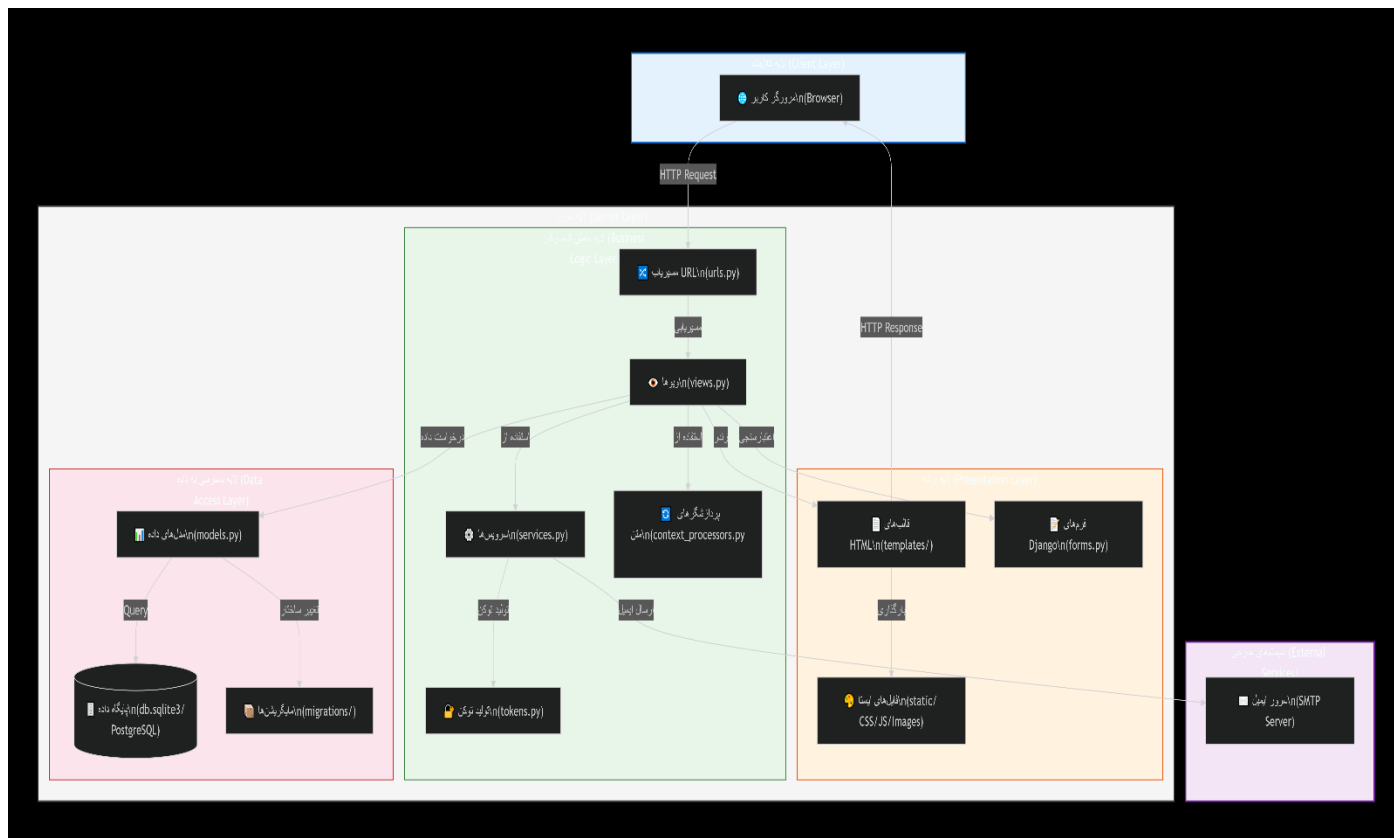
دسته	ابزار	کاربرد
مدیریت پروژه	GitHub Projects / Trello	مدیریت Sprint Backlog و Product Backlog
کنترل نسخه	GitHub (Git)	مدیریت کد، برنچ‌بندی، Pull Request
مستندسازی	GitHub	نگهداری مستندات پروژه
ارتباطات	Google Meet	ارتباط هفتگی تیم
ذخیره‌سازی کد	GitHub Repository	مخزن مرکزی کد

معماری و طراحی سیستم (Design & Architecture)

1. نمای کلی معماری (High-Level Architecture)

پروژه Stand Blog بر اساس معماری Monolithic Web Application با استفاده از فریم ورک Django پیاده‌سازی شده است. این معماری بر اساس الگوی MTV (Model-View-Template) یا همان MVC طراحی شده است.

دیاگرام معماری کلان (Comprehensive Architecture) :



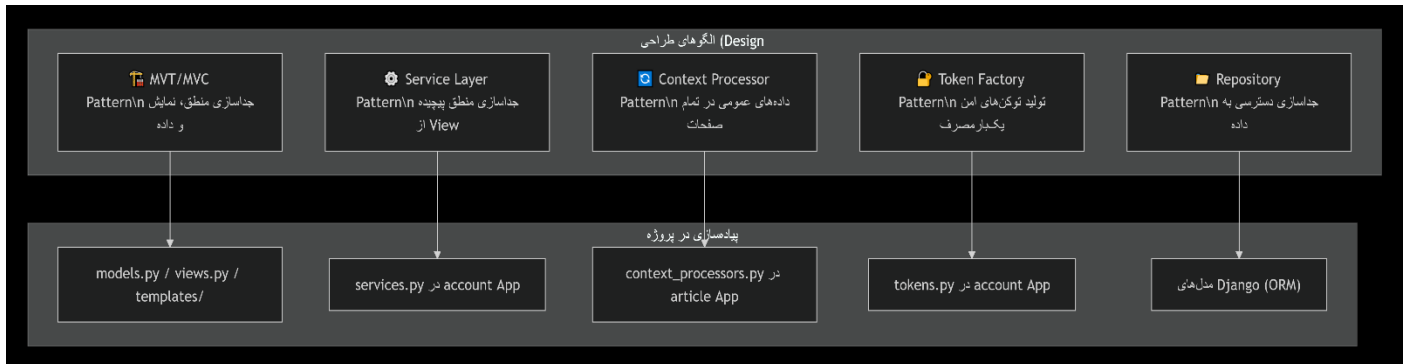
2. لایه‌های معماری (Architecture Layers)

جدول تفصیلی لایه‌ها:

لایه	مؤلفه‌ها	مسئولیت	فایل‌های مربوطه
لایه ارائه (Presentation)	Templates, Static, Forms	نمایش داده‌ها به کاربر، دریافت ورودی، اعتبارسنجی اولیه	templates/, static/, forms.py
لایه منطق کسب‌وکار (Business Logic)	Views, Services, Context Processors, Tokens	پردازش درخواست‌ها، اعمال قوانین کسب‌وکار، تولید توکن	views.py, services.py, context_processors.py, tokens.py
لایه دسترسی به داده (Data Access)	Models, Database, Migrations	ذخیره‌سازی و بازیابی داده‌ها، مدیریت ساختار دیتابیس	models.py, db.sqlite3, migrations/

3. الگوهای طراحی پیاده‌سازی شده (Design Patterns)

دیاگرام الگوهای طراحی

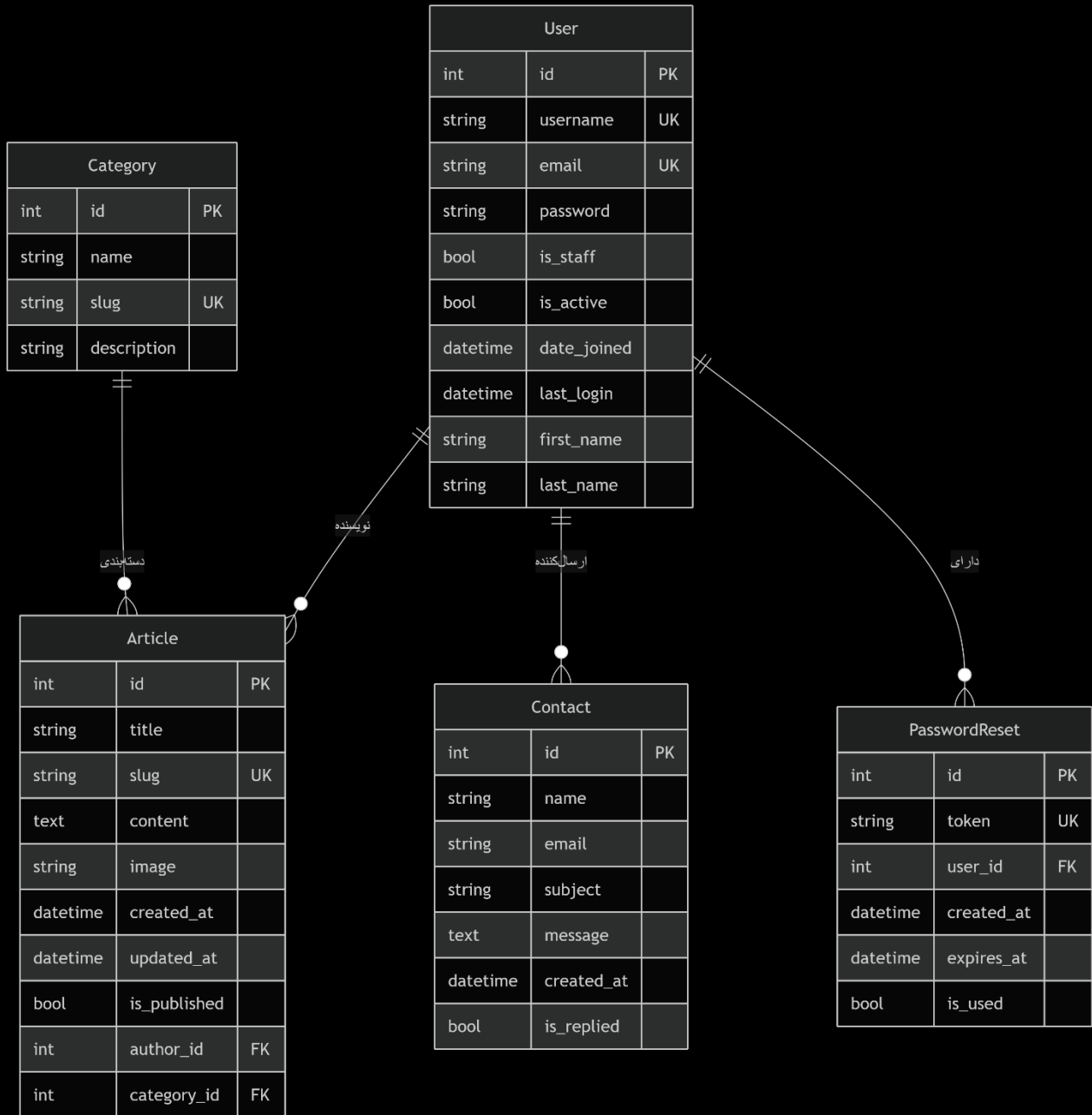


شرح کامل الگوها:

الگوی طراحی	محل پیاده‌سازی	مزیت
MVT/MVC	کل پروژه	جداسازی کامل لایه‌ها، افزایش قابلیت نگهداری
Service Layer	account/services.py	جداسازی منطق پیچیده احراز هویت از View ها، افزایش تست‌پذیری
Context Processor	article/context_processors.py	ارسال خودکار داده‌های عمومی (دسته‌بندی‌ها، آخرین مقالات) به تمام صفحات بدون تکرار کد
Token Factory	account/tokens.py	تولید توکن‌های یکبار مصرف امن با زمان انقضا برای بازیابی رمز
Repository	Django ORM (models.py)	جداسازی کوئری‌های دیتابیس از منطق برنامه

4. معماری پایگاه داده (Database Architecture)

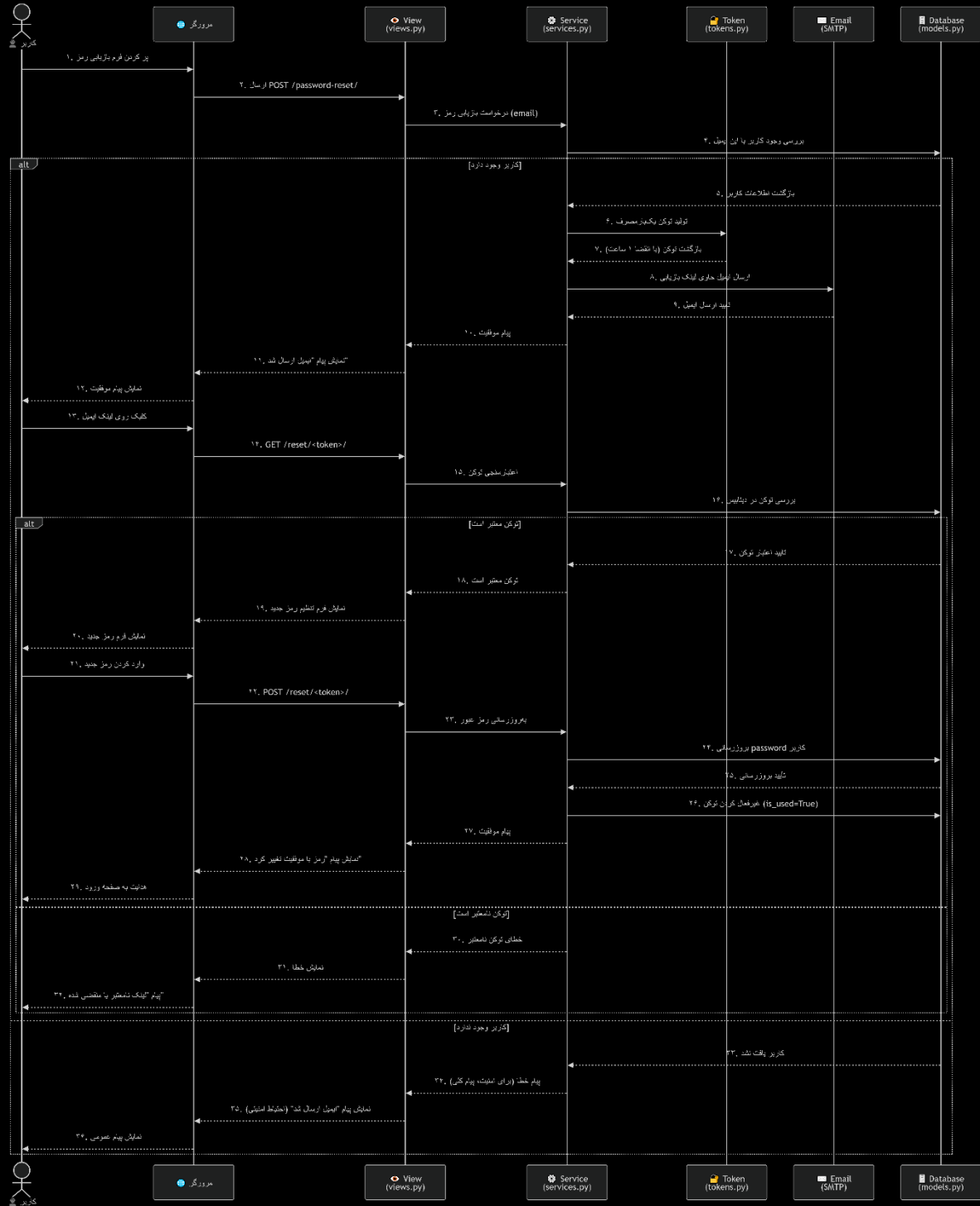
دیاگرام کامل (ERD (Entity-Relationship Diagram)



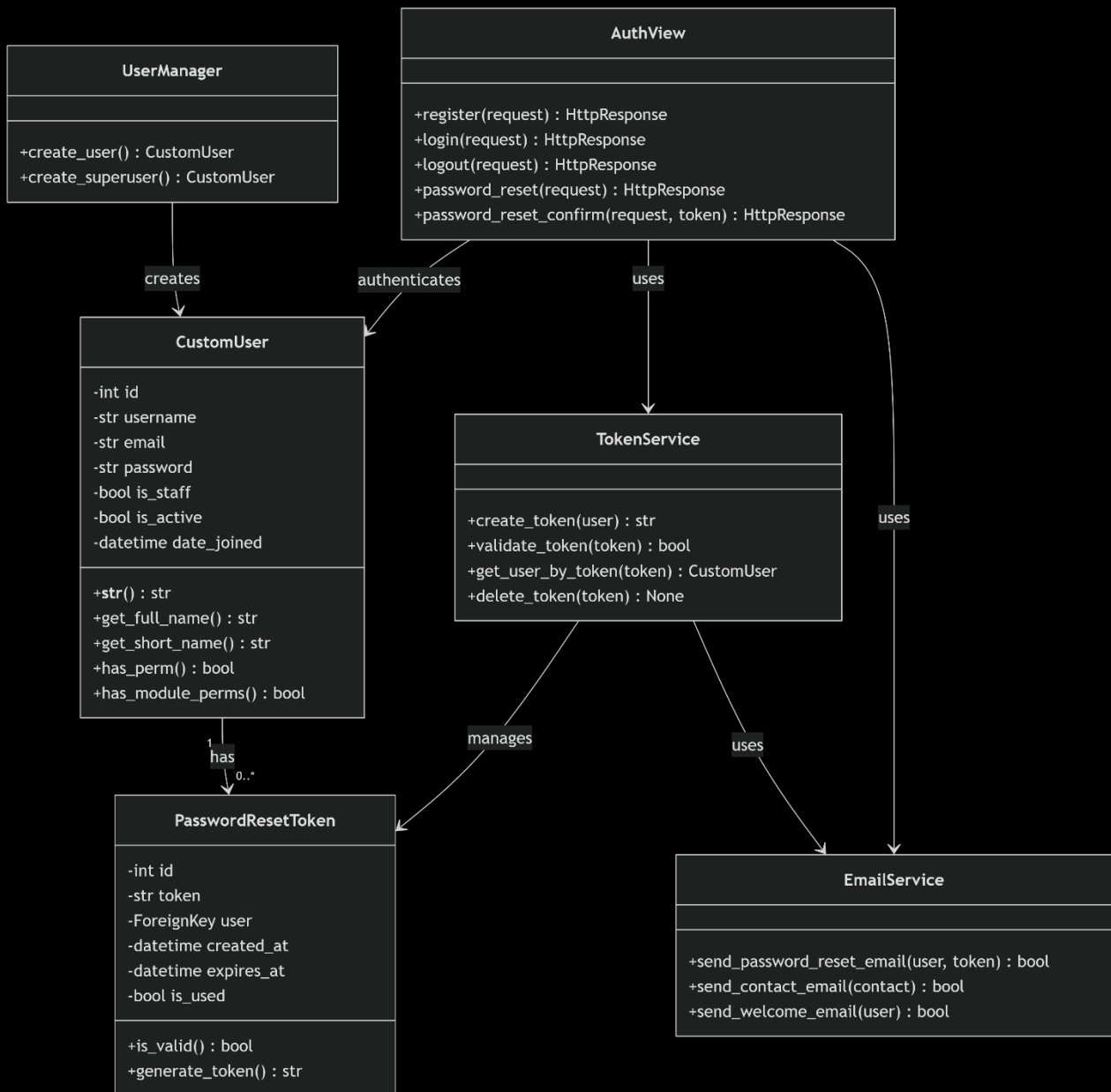
توضیحات جداول :

جدول	توضیح	فیلدهای کلیدی
User	مدل کاربر سفارشی (Custom User Model)	id, username, email, password, is_staff
Article	مقالات منتشر شده	id, title, slug, content, author_id
Category	دسته بندی مقالات	id, name, slug, description
Contact	پیام های فرم تماس	id, name, email, subject, message
PasswordReset	توکن های بازیابی رمز	id, token, user_id, expires_at

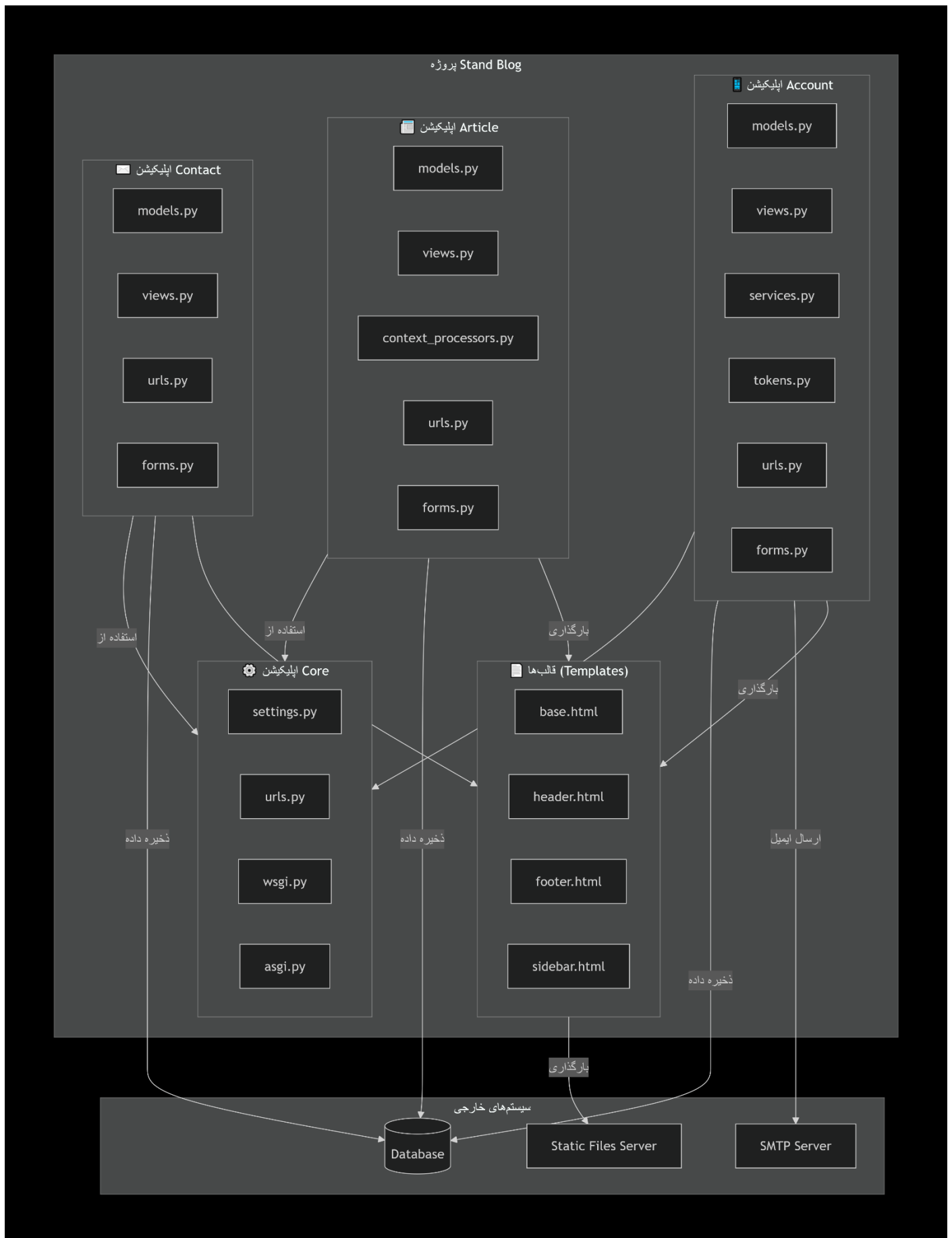
5. نمودار توالی - (Sequence Diagram)



6. نمودار کلاس ها - (Class Diagram) اپلیکیشن Account



7. نمودار مؤلفه‌ها (Component Diagram)



8. راهنمای کامل نصب و اجرا (Developer Manual)

پیشنیاز ها

توضیح	نسخه	نیاز
زبان برنامه‌نویسی	3.8+	Python
فریم‌ورک وب	4.2+	Django
کنترل نسخه	2.30+	Git
پایگاه داده	-	SQLite/PostgreSQL

مراحل نصب :

مرحله ۱: کلون کردن پروژه

کلون کردن ریپازیتوری

```
git clone https://github.com/your-team/stand-blog.git
```

ورود به پوشه پروژه

```
cd stand-blog
```

مرحله ۲: ایجاد و فعال‌سازی محیط مجازی

ایجاد محیط مجازی

```
python -m venv venv
```

فعال‌سازی در لینوکس/مک

```
source venv/bin/activate
```

فعال‌سازی در ویندوز

```
venv\Scripts\activate
```

مرحله ۳: نصب وابستگی‌ها

نصب تمام وابستگی‌ها از فایل `r.txt`

```
pip install -r r.txt
```

یا نصب دستی وابستگی‌های اصلی

```
pip install django==4.2
```

```
pip install django-crispy-forms
```

```
pip install pillow
```


مرحله ۴: تنظیم متغیرهای محیطی

ایجاد فایل .env در ریشه پروژه

```
echo "SECRET_KEY=your-secret-key-here" > .env
echo "DEBUG=True" >> .env
echo "EMAIL_HOST=smtp.gmail.com" >> .env
echo "EMAIL_PORT=587" >> .env
echo "EMAIL_HOST_USER=your-email@gmail.com" >> .env
echo "EMAIL_HOST_PASSWORD=your-app-password" >> .env
```

مرحله ۵: اجرای مایگریشن‌ها

ساخت مایگریشن‌ها

```
python manage.py makemigrations
```

اعمال مایگریشن‌ها

```
python manage.py migrate
```

مرحله ۶: ایجاد کاربر ادمین

ایجاد سوپر یوزر برای دسترسی به پنل ادمین

```
python manage.py createsuperuser
```

مرحله ۷: جمع‌آوری فایل‌های ایستا

جمع‌آوری فایل‌های استاتیک (CSS, JS, Images)

```
python manage.py collectstatic
```

مرحله ۸: اجرای سرور توسعه

اجرای سرور روی پورت 8000

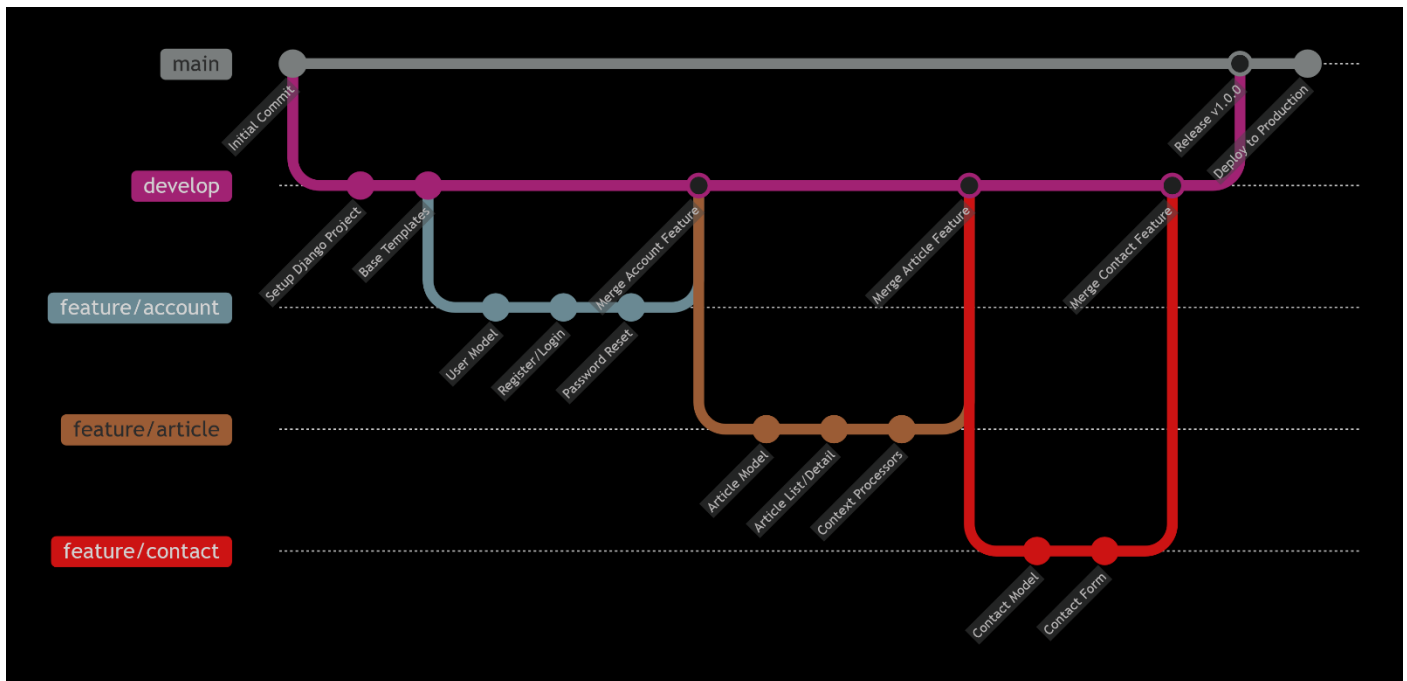
```
python manage.py runserver
```

یا روی پورت خاص

```
python manage.py runserver 8080
```

9. ساختار برنچ‌های Git (Git Branching Strategy)

دیاگرام استراتژی برنچ‌ها (Git Flow)



شرح برنچ‌ها:

برنچ	توضیح	قوانین
main	کد نهایی و پایدار	فقط از develop Merge می‌شود، ممنوعیت Commit مستقیم
develop	برنچ اصلی توسعه	تمام Feature برنچ‌ها از اینجا جدا می‌شوند
feature/*	توسعه ویژگی‌های جدید	از develop جدا می‌شوند و به develop Merge می‌شوند
hotfix/*	رفع باگ‌های بحرانی	از main جدا می‌شوند و به main و develop Merge می‌شوند
release/*	آماده‌سازی نسخه جدید	از develop جدا می‌شوند و به main Merge می‌شوند

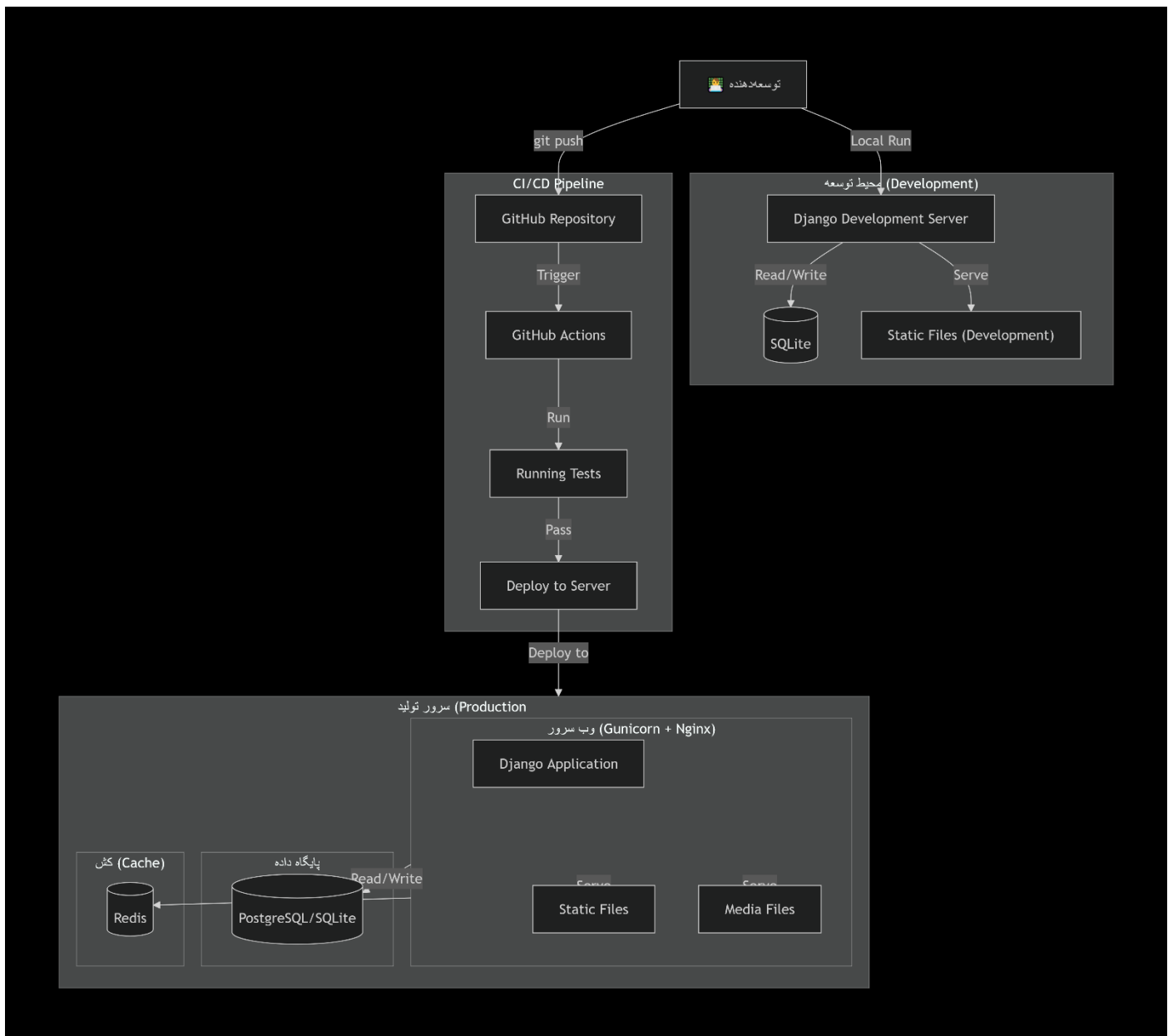
گردش کار: (Workflow)

1. شروع توسعه/ git checkout -b feature نام-ویژگی develop
2. توسعه Commit: های منظم با پیام های توصیفی
3. پایان توسعه git checkout develop و: git merge feature نام-ویژگی
4. حذف برنج/ git branch -d feature نام-ویژگی
5. انتشار نسخه git checkout main و: git merge develop

10 . مستندات API

مقد	مسیر (Endpoint)	توضیح	ورودی	خروجی
POST	/api/register/	ثبت نام کاربر جدید	{username, email, password}	{message, user_id}
POST	/api/login/	ورود کاربر	{username, password}	{message, token}
POST	/api/password-reset/	درخواست بازیابی رمز	{email}	{message}
POST	/api/password-reset/confirm/	تأیید بازیابی رمز	{token, new_password}	{message}
GET	/api/articles/	دریافت لیست مقالات	{page, category}	{articles, pagination}
GET	/api/articles/{slug}/	دریافت جزئیات مقاله	{slug}	{article, author}
POST	/api/contact/	ارسال فرم تماس	{name, email, subject, message}	{message}

Deployment Diagram .11



خلاصه معماری

جنبه معماری	توضیح
الگوی معماری	Monolithic Web Application (Django MTV)
لایه‌ها	Presentation, Business Logic, Data Access
الگوهای طراحی	MVT, Service Layer, Context Processor, Token Factory, Repository
پایگاه داده	(SQLite توسعه) / PostgreSQL (تولید)
مدیریت نسخه	Git Flow (main, develop, feature/*)
ابزارهای تست	Django Test Framework, Pytest
استقرار	Gunicorn + Nginx